Things Fall Apart: Navigating Managed Databases for Over a Decade as a Non-DBA

•••

Renato Losio

My (RDS) journey so far

AWS News Blog

Introducing Amazon RDS – The Amazon Relational Database Service

by Jeff Barr | on 26 OCT 2009 | in Amazon RDS | Permalink | Share



Voiced by Amazon Polly

We are always looking for ways to make it faster, simpler, and more fun to develop applications of all types. Every hour that you don't spend fiddling with hardware, tracing cables, installing operating systems or managing databases is an hour that you can spend on the unique and value-added aspects of your application.

Today I'd like to tell you about our newest service, the Amazon Relational Database Service, or Amazon RDS for short. Now in beta, RDS makes it easier for you to set up, operate, and scale a relational database in the cloud. You get direct database access without worrying about infrastructure provisioning, software maintenance, or common database management tasks.

Using the RDS APIs or the command-line tools, you can access the full capabilities of a complete, self-contained MySQL 5.1 database instance in a matter of minutes. You can scale the processing power and storage space as needed with a single API call and you can initiate fully consistent database snapshots at any time.

Back in 2011

Simplicity

Constraint

More expensive



Announcing Multi-AZ Deployments for Amazon RDS

Posted On: May 18, 2010

We are excited to announce Multi-Availability Zone (Multi-AZ) deployments for Amazon Relational Database Service (Amazon RDS). This new deployment option provides enhanced availability and data durability by automatically replicating database updates between multiple Availability Zones. Availability Zones are physically separate locations with independent infrastructure engineered to be insulated from failure in other Availability Zones. When you create or modify your DB Instance to run as a Multi-AZ deployment, Amazon RDS will automatically provision and maintain a synchronous "standby" replica in a different Availability Zone. In the event of planned database maintenance or unplanned service disruption, Amazon RDS will automatically failover to the up-to-date standby so that database operations can resume quickly without administrative intervention.

The increased availability and fault tolerance offered by Multi-AZ deployments are well suited to critical production environments. To learn more, visit the Amazon RDS product page.

m1.xlarge	0.465
m2.xlarge	0.330
m3.xlarge	0.370
m4.xlarge	0.350
m5.xlarge	0.342
m6i.xlarge	**0.342**
m6g.xlarge	**0.304**
m7g.xlarge	**0.337**
t3.xlarge	**0.272**
t4g.xlarge	**0.258**

GP2 (or GP3)

MariaDB

Oracle

Provisioned
IOPS

SQLServer

RDS without

PostgreSQL

Amazon
Aurora

PerformanceInsights

Iterate, and keep up to date

Some interesting CASEs

Learn through Support

[CASE *****1] Multi AZ replica broken after scale up from m7g.2xlarge to m7g.4xlarge

[CASE *****2] Aurora MySQL minor roadmap versus RDS

[CASE *****3] Enable Amazon RDS Optimized Writes using RDS Blue/Green Deployments

[CASE *****4] RDS stuck in Storage-optimization 99% for almost 24 hours

[CASE *****5] System snapshot creation time exploded without significant (...)

[CASE *****6] Support for innodb_redo_log_capacity?

[CASE *****7] No way to delete old automatic snapshot outside backup period

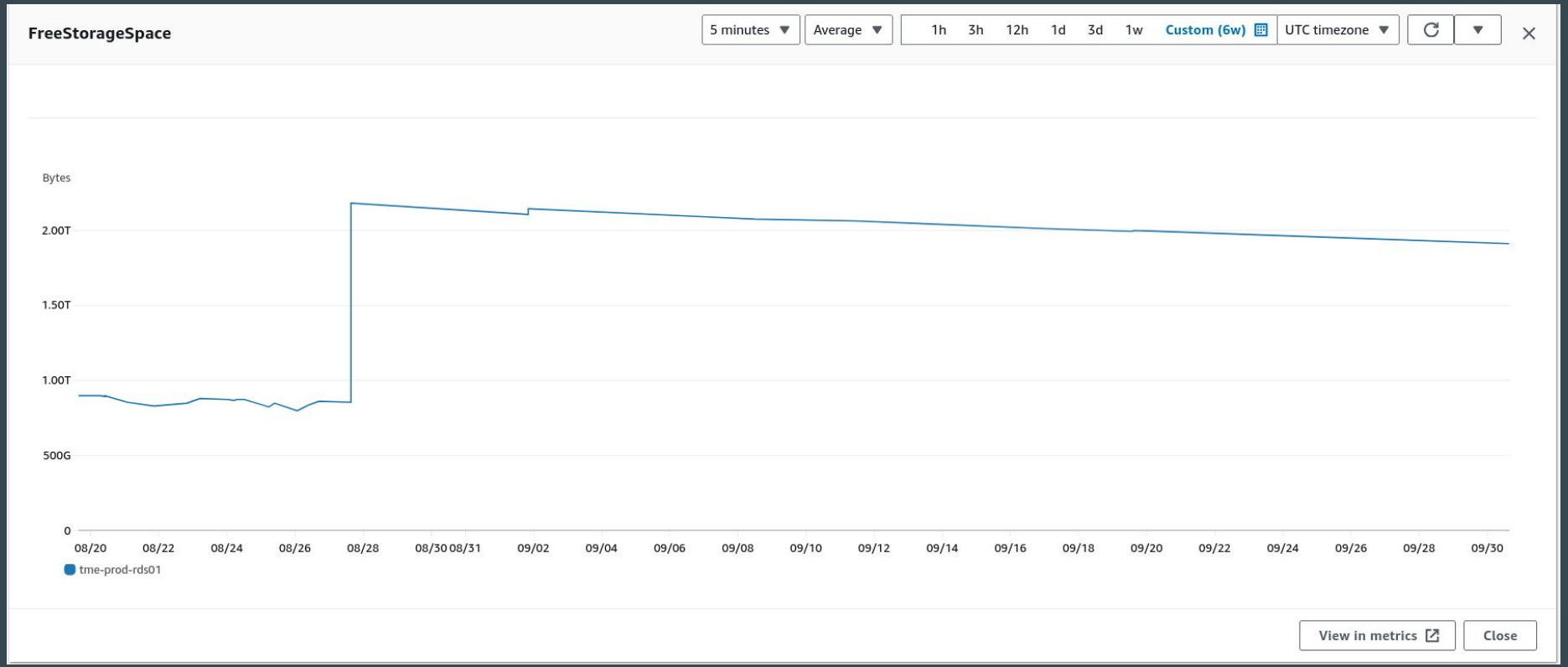
[CASE *****8] binlog_transaction_compression + binlog_transaction_compression_level_zstd

[CASE *****9] RDS, Blue/Green and backup retention

[CASE *****0] mysqld crash with optimize table statement

Challenges and solutions

How much free storage you need?



1) Storage autoscaling

```
OPTIMIZE TABLE MyLargeTable;

ALTER TABLE MyLargeTable MODIFY ENUM myEnum(U,A,B,D);
```



```
EVENTS  2024-08-25T15:48:40.477000+00:00        Applying autoscaling-initiated
modification to allocated storage.
arn:aws:rds:eu-west-1:053652639***:db:***-rds01-read-replica
tme-prod-rds01-read-replica  db-instance
```

How to ALTER/OPTIMIZE an InnoDB table?

How?

- Native ALTER TABLE
- Third-party tool (pt-online-schema-change, etc.)

Where?

- Primary node
- Replica node (stop + swap)
- Vendor specific (Amazon RDS Blue/Green)



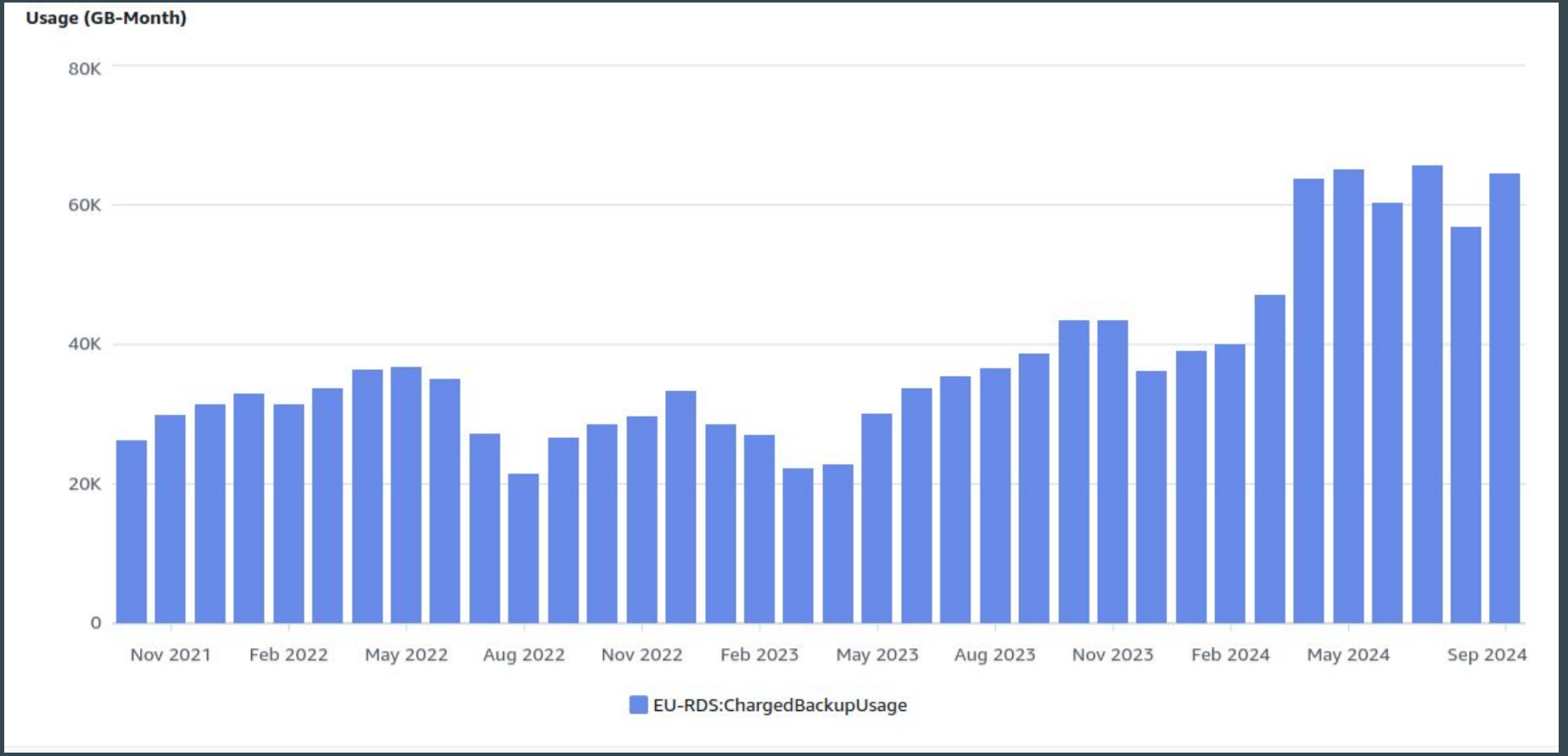
There is no additional charge for backup storage, up to 100% of your total database storage for a region. (Based upon our experience as database administrators, the vast majority of databases require less raw storage for a backup than for the primary dataset, meaning that most customers will never pay for backup storage.)

(AWS Documentation)

2) CPU is overestimated

Monthly storage and backup storages grow over time

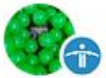
USD/Month	**2022**	**2023**	**2024**
CPU (RI)	3336	4682	6006
Storage (GP)	1960	2387	2994
Backup	2274	3300	5480


Usage (GB-Month)
80K
60K
40K
20K
0
Nov 2021
Feb 2022
May 2022
Aug 2022
Nov 2022
Feb 2023
May 2023
Aug 2023
Nov 2023
Feb 2024
May 2024
Sep 2024
EU-RDS:ChargedBackupUsage

Amazon RDS supports ARM or x86 processors? Good for you, you're a freaking database service. I couldn't possibly care less about that; just run my workload and stop bothering me with trivia.

Corey Quinn

Managing auto-scaling of RDS on AWS using the AWS CLI and Bash



Renato Losio

Published in funambol-techblog · 6 min read · Aug 1, 2019

31 1

Cloud services enable you to increase or decrease the capacity of your deployment within minutes, but **how can you scale a relational database automatically keeping predictable performance and costs under control?**

In this post we will show how Funambol manages the vertical scale up and scale down of all its MySQL databases running on RDS and handles peak traffic using the AWS Command Line Interface, AWS CloudWatch alarms and some simple logic that can be even written in a Bash script.

3) How I crashed my ARM-based cluster

```
Most likely, you have hit a bug, but this error can also be caused by
malfunctioning hardware.
Thread pointer: 0x4017ad88f800
Attempting backtrace. You can use the following information to find out
where mysqld died. If you see no messages after this, something went
terribly wrong...
stack_bottom = 4017f3b1ee60 thread_stack 0x40000
/rdsdbbin/mysql/bin/mysqld(my_

print_stacktrace(unsigned char const*, unsigned long)+0x30) [0x1f7bb30]
/rdsdbbin/mysql/bin/mysqld(print_fatal_signal(int)+0x27c) [0xfaa93c]
/rdsdbbin/mysql/bin/mysqld(handle_fatal_signal+0xc4) [0xfaaa64]
linux-vdso.so.1(__kernel_rt_sigreturn+0) [0x400014b92860]
/rdsdbbin/mysql/bin/mysqld(trx_undo_read_v_cols(dict_table_t const,
```

Parameters comparison

Parameter	my-questionable-parameters	default.mysql8.0
event_scheduler	ON	
general_log	0	
innodb_buffer_pool_size	{DBInstanceClassMemory*11/16}	{DBInstanceClassMemory*3/4}
innodb_change_buffering	all	
innodb_flush_log_at_trx_commit	2	
innodb_log_file_size	21474836480	134217728
innodb_online_alter_log_max_size	1342177280000	
innodb_print_all_deadlocks	1	
innodb_redo_log_capacity	21474836480	2147483648
log_bin_trust_function_creators	1	
max_allowed_packet	64000000	
max_connections	5000	{DBInstanceClassMemory/12582880}
max_execution_time	180000	
range_optimizer_max_mem_size	8388608	
slave_parallel_workers	1	
slow_query_log	1	
table_open_cache	10000	

Migrate from RDS MySQL to Aurora MySQL in near zero downtime

Introduction

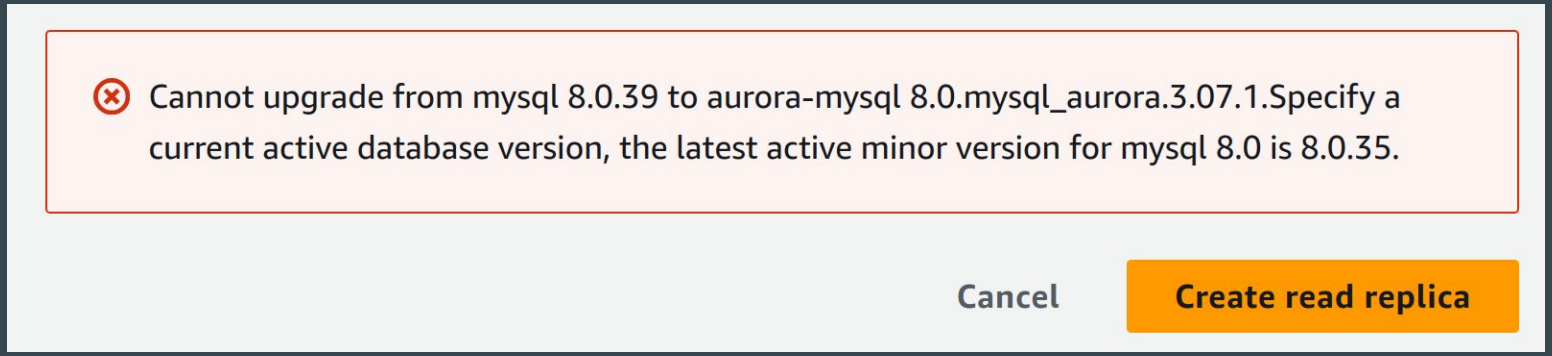
In this tutorial, we will look at how to migrate from Amazon RDS MySQL to Amazon Aurora MySQL with minimal downtime. As with any database migration, there are several options. For migrating data from a MySQL DB Instance to an Amazon Aurora MySQL DB Cluster, we recommend to use a special type of node called an Aurora Read Replica for the source MySQL DB instance. Amazon RDS uses the MySQL DB engines' binary log replication functionality and updates made to the source MySQL DB instance are asynchronously replicated to the Aurora Read Replica. As replication lag between source DB instance and Aurora Read Replica approaches zero, redirect your client applications to the Aurora Read Replica, and make the Aurora Read Replica a standalone Aurora MySQL DB cluster. For more information, refer Amazon Aurora documentation.

The methodology and steps discussed in this tutorial are applicable to any application's database that resides on Amazon RDS MySQL DB instance and needs to be migrated to Aurora MySQL DB cluster.

The tutorial is not within the free tier and will cost you less than $1 provided you follow the steps in the tutorial and terminate your resources at the end of the tutorial.

✔ **AWS Experience**	Intermediate
Time to Complete	10 - 20 minutes
$ Cost to Complete	Less than $1
Services Used	AWS RDS AWS Aurora

4) Going too fast. And missing out.



My RDS is running out of IOPS. What can I do?

January 31, 2017

One of the hardest challenges to handle with RDS is running out of IOPS.

How RDS storage works

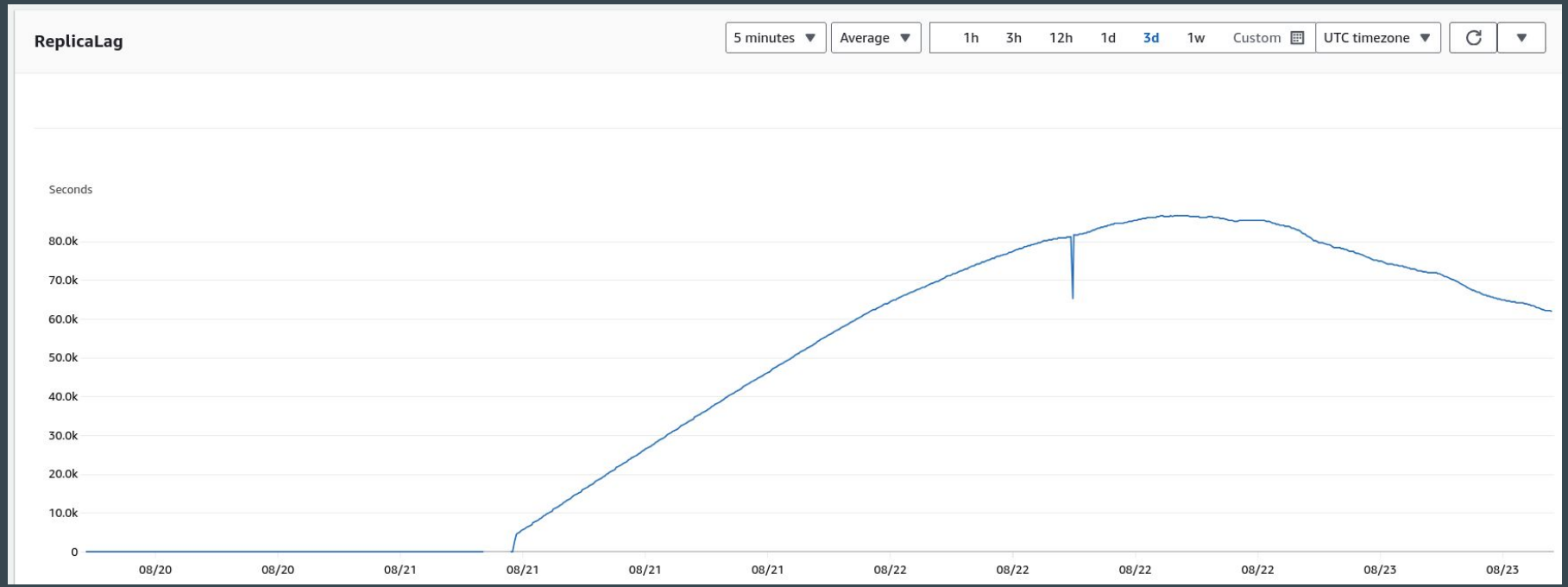
If you are not already familiar with the topic, there is a very detailed Storage for Amazon RDS page that covers the different storage options. The GP2 volumes have the base performance 3 times of their allocated size. For example a 200GB RDS will have a baseline of 600 IOPS, a 1TB RDS will have a baseline of 3000 IOPS. In case you temporary need more IOPS, the GP2 volumes will give you a Burst Balance up to a maximum 3000 IOPS. When the Burt Balance is empty, you go down to the base performance (for

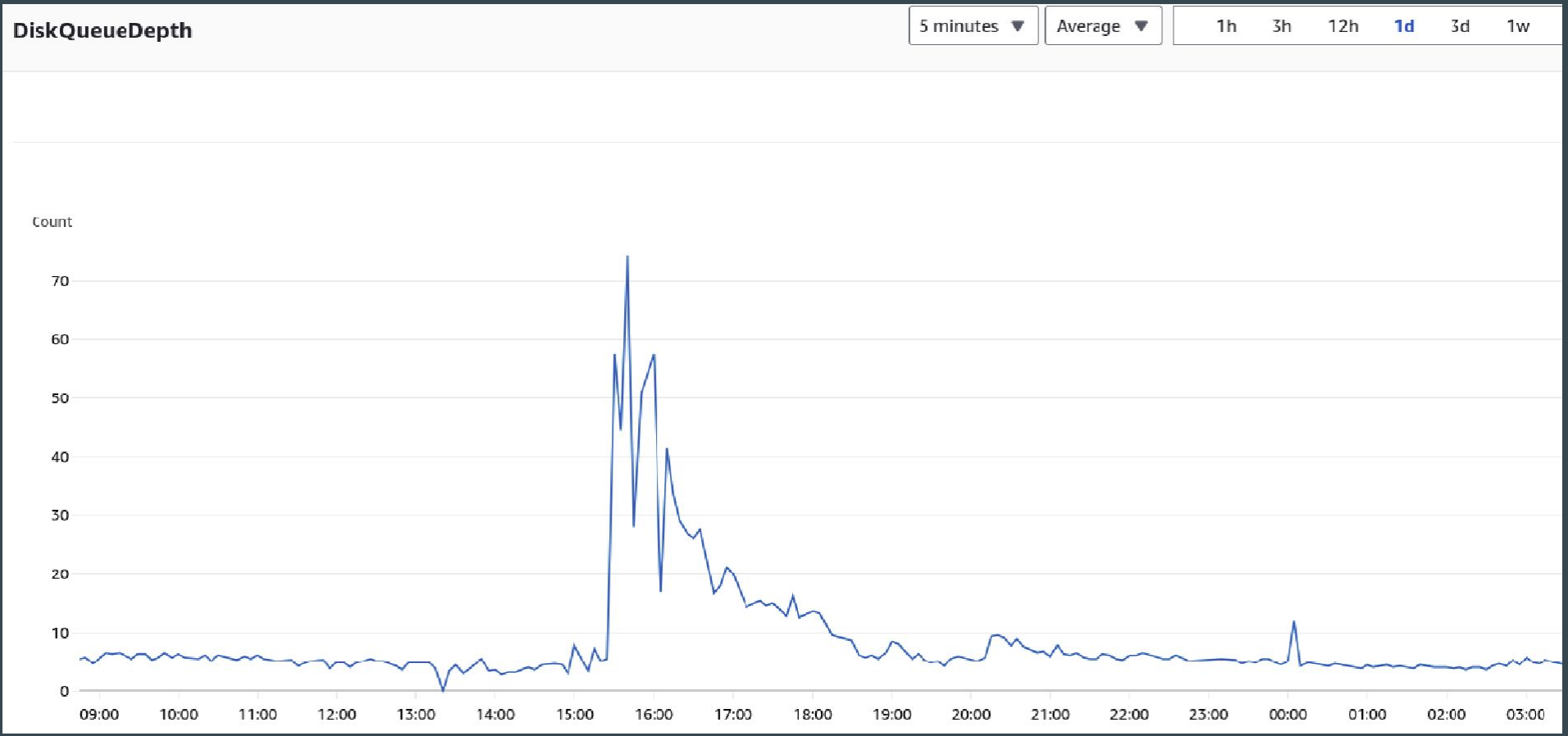
5) Running out of IOPS

- You always underestimate IOPS
- The (newish) Total IOPS metric is useful
- **gp3** is usually better than **gp2**
- **gp3** is always more flexible than **gp2**
- do not use io1 on latest engines
- **io2** Express way better but often not needed
- watched out for copies, replicas, tests and staging instances

6) Underestimate time for a change

How long does an ALTER TABLE require promoting a replica?




DiskQueueDepth
5 minutes
Average
1h
3h
12h
1d
3d
1w
Count
70
60
50
40
30
20
10
0
09:00
10:00
11:00
12:00
13:00
14:00
15:00
16:00
17:00
18:00
19:00
20:00
21:00
22:00
23:00
00:00
01:00
02:00
03:00

Super simple experiments

Benchmarking basics

- Aurora (MySQL) versus RDS for MySQL
- RDS for MySQL versus RDS for MariaDB

```
RDS setup: 400 GB + db.r7g.large
Aurora setup: db.r7g.large
```

Inserting 100K records

- One at a time
- Single thread
- ACID

CPU

Developer

Engine

it depends...

Database

Table

Version

Latency

Storage

Memory

SQL

```
CREATE TABLE aws_community_day
(id bigint(20) NOT NULL AUTO_INCREMENT,
datetime TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
value integer DEFAULT NULL,
PRIMARY KEY (id));


DELIMITER $$
CREATE PROCEDURE load_demo()
BEGIN
  DECLARE i INT DEFAULT 0;
  WHILE i < 100000 DO
    INSERT INTO aws_community_day (value) VALUES (FLOOR(RAND()*1000));
    SET i = i + 1;
  END WHILE;
END$$
DELIMITER ;

CALL load_demo();
```

Minutes?

Seconds?

how long?

(db.m7g.large)

Milliseconds?

Hours?

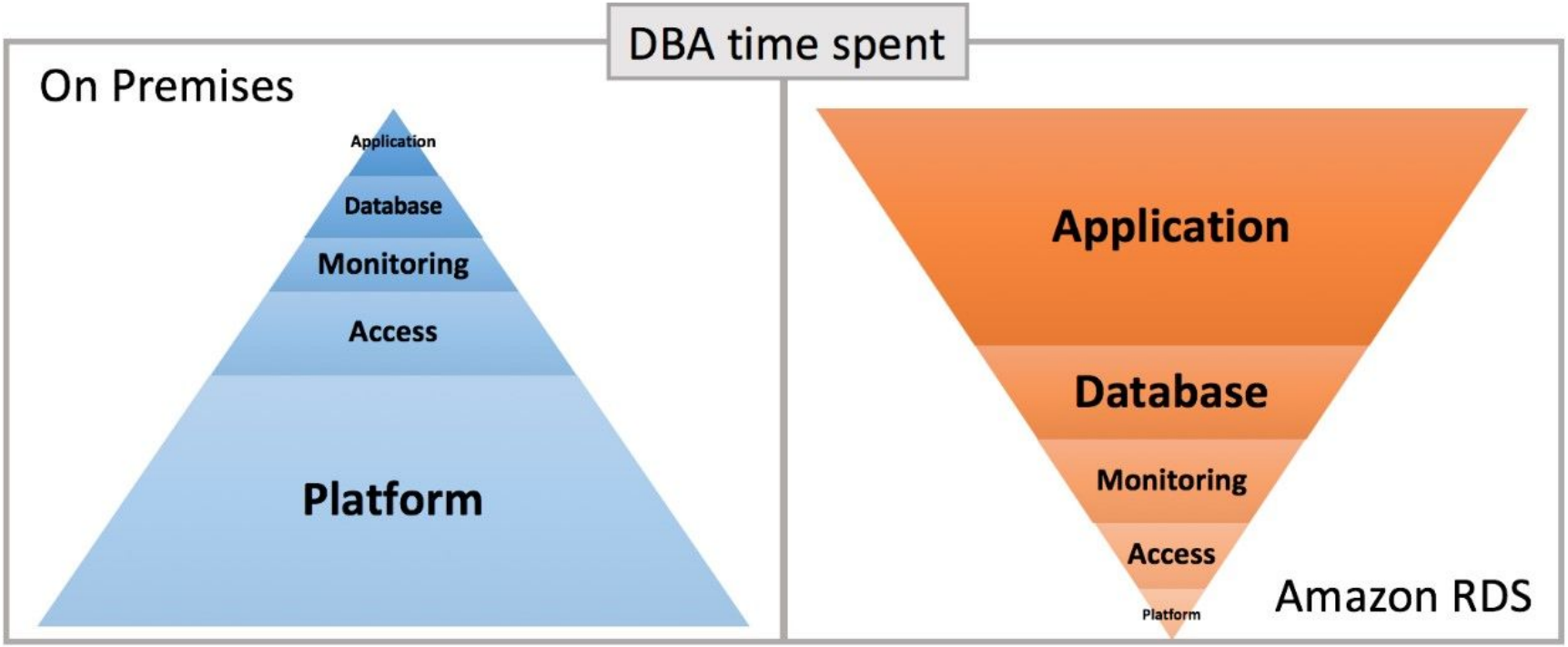
Aurora vs. Amazon RDS for MySQL

- Amazon RDS 8.0.39 (5m 49s)
- Aurora 3.05.2 (2.7s)

RDS for MySQL vs. RDS for MariaDB

- MySQL 8.0.39 (5m 49s)
- MariaDB 10.11.8 (5m 48s)

Are you really not a DBA?


DBA time spent
On Premises
Application
Database
Monitoring
Access
Platform
Application
Database
Monitoring
Access
Platform
Amazon RDS

Lessons learned (1/2)

- Changes to allocated storage might take hours or days. Plan accordingly.
- You might play with a managed service but logs, performance schema, and parameters are all available. Look under the hood.
- Avoid changing default settings for short term gains. You might need to scale automatically your instance later.

Lessons learned (2/2)

- Validate your findings and your plans with the support team. They have access to your database.
- Warm up your database replica before directing traffic to it, the storage might still be cold.
- Cost optimization is vital but don’t focus only on the instance size, engine, or service. Storage and backup matter.
- Approach benchmarks cautiously.

Leverage managed database, resist changes, but understand what runs under the hood.

Thank you!

renato@cloudiamo.com